

Integral

数值积分

定积分的定义

简单来说，函数 $f(x)$ 在区间 $[a, b]$ 上的定积分 指的是 在区间 $[a, b]$ 中与 x 轴围成的区域的面积（其中 x 轴上方的部分为正值， x 轴下方的部分为负值）。

很多情况下，我们需要高效，准确地求出一个积分的近似值。下面介绍的 **辛普森法**，就是这样一种求数值积分的方法。

辛普森法

这个方法的思想是将被积区间分为若干小段，每段套用二次函数的积分公式进行计算。

► 二次函数积分公式（辛普森公式）

根据这个辛普森公式，我们先介绍一种普通的辛普森积分法。

普通辛普森法

1743 年，这种方法发表于托马斯·辛普森的一篇论文中。

描述

给定一个自然数 n ，将区间 $[a, b]$ 分成 n 个等长的区间。

我们就可以计算每个小区间 $[x_{i-1}, x_i]$ 的积分值，将所有区间的积分值相加即为总积分。

对于 $[x_{i-1}, x_i]$ 的一个区间，选其中的三个点 $x_{i-1}, x_i, x_{i-1/2}$ 就可以构成一条抛物线从而得到一个函数 $q(x)$ ，这个函数存在且唯一。计算原函数 $f(x)$ 在该区间的积分值就变成了计算新的二次函数 $q(x)$ 在该段区间的积分值。这样我们就可以利用辛普森公式来近似计算它。

将其分段求和即可得到如下结论：

误差

我们直接给出结论，普通辛普森法的误差为：

其中 ξ 是位于区间 $[a, b]$ 的某个值。

实现



```

1 constexpr int N = 1000 * 1000;
2
3 double simpson_integration(double a, double b) {
4     double h = (b - a) / N;
5     double s = f(a) + f(b);
6     for (int i = 1; i <= N - 1; ++i) {
7         double x = a + h * i;
8         s += f(x) * ((i & 1) ? 4 : 2);
9     }
10    s *= h / 3;
11    return s;
12 }

```

```

1 N = 1000 * 1000
2
3
4 def simpson_integration(a, b):
5     h = (b - a) / N
6     s = f(a) + f(b)
7     for i in range(1, N):
8         x = a + h * i
9         if i & 1:
10            s = s + f(x) * 4
11        else:
12            s = s + f(x) * 2
13    s = s * (h / 3)
14    return s

```

自适应辛普森法

普通的方法为保证精度在时间方面无疑会受到 的限制，我们应该找一种更加合适的方法。

现在唯一的问题就是如何进行分段。如果段数少了计算误差就大，段数多了时间效率又会低。我们需要找到一个准确度和效率的平衡点。

我们这样考虑：假如有一段图像已经很接近二次函数的话，直接带入公式求积分，得到的值精度就很高了，不需要再继续分割这一段了。

于是我们有了这样一种分割方法：每次判断当前段和二次函数的相似程度，如果足够相似的话就直接代入公式计算，否则将当前段分割成左右两段递归求解。

现在就剩下一个问题了：如果判断每一段和二次函数是否相似？

我们把当前段直接代入公式求积分，再将当前段从中点分割成两段，把这两段再直接代入公式求积分。如果当前段的积分和分割成两段后的积分之和相差很小的话，就可以认为当前段和二次函数很相似了，不用再递归分割了。

上面就是自适应辛普森法的思想。在分治判断的时候，除了判断精度是否正确，一般还要强制执行最少的迭代次数。

参考代码如下：



C++Python

```

1 double simpson(double l, double r) {
2     double mid = (l + r) / 2;
3     return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6; // 辛普森公式
4 }
5
6 double asr(double l, double r, double eps, double ans, int step) {
7     double mid = (l + r) / 2;
8     double fl = simpson(l, mid), fr = simpson(mid, r);
9     if (abs(fl + fr - ans) <= 15 * eps && step < 0)
10         return fl + fr + (fl + fr - ans) / 15; // 足够相似的话就直接返回
11     return asr(l, mid, eps / 2, fl, step - 1) +
12           asr(mid, r, eps / 2, fr, step - 1); // 否则分割成两段递归求解
13 }
14
15 double calc(double l, double r, double eps) {
16     return asr(l, r, eps, simpson(l, r), 12);
17 }

1 def simpson(l, r):
2     mid = (l + r) / 2
3     return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6 # 辛普森公式
4
5
6 def asr(l, r, eps, ans, step):
7     mid = (l + r) / 2
8     fl = simpson(l, mid)
9     fr = simpson(mid, r)
10    if abs(fl + fr - ans) <= 15 * eps and step < 0:
11        return fl + fr + (fl + fr - ans) / 15 # 足够相似的话就直接返回
12    return asr(l, mid, eps / 2, fl, step - 1) + asr(
13        mid, r, eps / 2, fr, step - 1
14    ) # 否则分割成两段递归求解
15
16
17 def calc(l, r, eps):
18    return asr(l, r, eps, simpson(l, r), 12)

```

习题

- [Luogu4525【模板】自适应辛普森法 1](#)
- [HDU1724 Ellipse](#)
- [NOI2005 月下柠檬树](#)

参考资料

<https://doi.org/10.1145/321526.321537>: 该文章讨论了自适应 Simpson 法的改进方案, 其中详细论述了上代码中的常数 15 的由来与优势。

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [H-J-Granger](#), [Chrogeek](#), [countercurrent-time](#), [Enter-tainer](#), [Great-designer](#), [iamtwz](#), [Ir1d](#), [ksyx](#), [mao1t](#), [Menci](#), [NachtgeistW](#), [Nanarikom](#), [ShaoChenHeng](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [shawlleyw](#), [YanJun-Zhao](#), [zyj-111](#)

